

≡ Hide menu

About this Course

Key Components of Llamafile

Developing Portable CLI with Cosmopolitan

Running Llamafile

✓ **Reading:** Key Terms
5 min

📖 **Reading:** What are LLMs?
5 min

✓ **Video:** Getting Started with Llamafile
3 min

✓ **Video:** Llamafile local system metrics
3 min

📅 **Lab:** Local Llamafile API
10 min

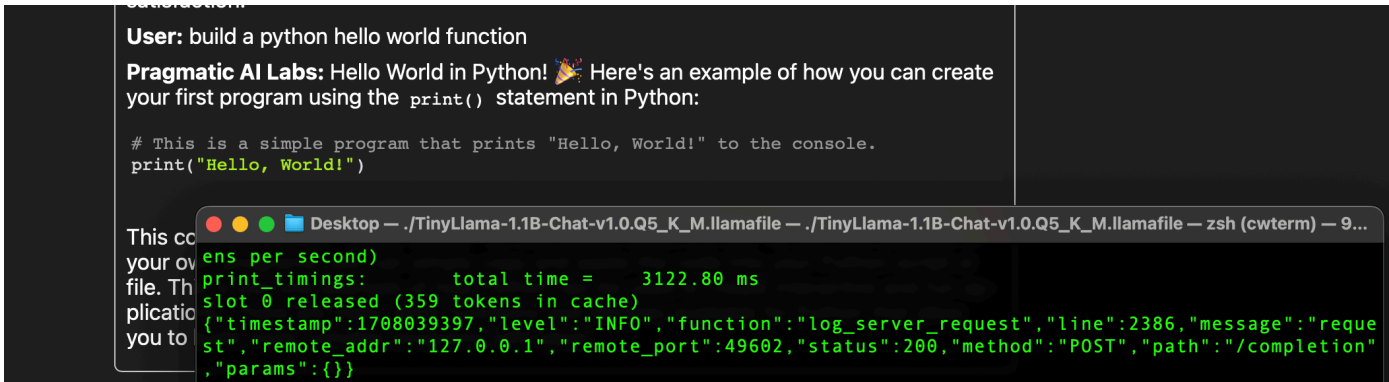
✓ **Graded Assignment:** Quiz- Running Llamafile
Submitted

📖 **Reading:** Lesson Reflection
5 min

Conclusion and Next Steps

Key Terms

Llamafile - A single executable file that contains all the code and models needed to run a large language model locally. Enables privacy and good performance.



mixtral - A high quality 30GB open source language model that can be run with llama file. Comparable performance to commercial models.

cosmopolitan - The library used by llama file to package everything into a single executable.

LLaVA - An open source model that can analyze images. Can be run with llama file.

Elo rating - A system to quantify the relative skill level of language models.

```
1 import math
2 def calculate_elo(player_elo, opponent_elo, scored, expected):
3     """
4     Calculate new Elo rating for a player
5     """
6     k_factor = 32
7
8     elo_diff = opponent_elo - player_elo
9     expected_score = 1 / (1 + 10 ** (elo_diff / 400))
10    new_elo = player_elo + k_factor * (scored - expected_score)
11
12    return new_elo
13
14    player_elo = 1500
15    opponent_elo = 1600
16    scored = 1 #Player won
17    expected_score = 1 / (1 + 10 ** ((opponent_elo - player_elo) / 400))
18
19    new_elo = calculate_elo(player_elo, opponent_elo, scored, expected_score)
20
21    print(f"New Elo rating: {new_elo}")
22
```

Run

Reset

New Elo rating: 1520.4820799936924

cURL - A command line tool that makes HTTP requests and transfers data using URL syntax. Allows interacting with web APIs and servers from a terminal/shell instead of browser.

Some key features of cURL:

- Supports common HTTP methods like GET, POST, PUT, DELETE out of the box
- Easy to customize requests by adding headers, authentication, proxies
- Can send form data, JSON, custom bodies
- Upload/download files from remote servers
- Follow redirects and store response info
- Available on virtually all platforms - Linux, Windows, MacOS

For example, to make a GET request with cURL:

```
1 curl https://www.example.com
```

And to POST JSON data:

```
1 curl -d '{"key":"value"}' -H "Content-Type: application/json" https://api.example.com
```

API Endpoint - The URL that defines the resource and function being requested. In this case `http://localhost:8080/v1/chat/completions` is the endpoint to get AI-generated text completions.

Official Llamafile curl example

```
1 curl http://localhost:8080/v1/chat/completions \
2 -H "Content-Type: application/json" \
3 -H "Authorization: Bearer no-key" \
4 -d '{
5   "model": "LLaMA_CPP",
6   "messages": [
7     {
8       "role": "system",
9       "content": "You are LLAMAFile, an AI assistant. Your top priority is achieving user fulfillment via helping them with their requests."
10    },
11    {
12       "role": "user",
13       "content": "Write a limerick about python exceptions"
14    }
15  ]
16 }' | python3 -c '
17 import json
18 import sys
19 json.dump(json.load(sys.stdin), sys.stdout, indent=2)
20 print()
21 '
```

Authorization Header - Specifies authentication info so the user/client can access the API. Here we pass a dummy bearer token that gives public access.

JSON Payload - The input data passed to the API, formatted as JSON. Includes parameters like the AI model and chat messages with roles.

Python Deserialization - Converts the JSON response text into Python data structures to make it easier to process programmatically.

Chat Message - A single dialogue turn passed to the AI to provide context. Has properties like the speaker role and utterance content.

Chat Context - The full chat history and assistant instructions that inform how the AI will respond and generate text.

Go to next item ✓ Completed

👍 Like 👎 Dislike 🐞 Report an issue